

Chapter 7: Reflection and Self-Critique Pattern for Adaptive Agent Improvement

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 07

File Name: chapter-07-reflection-self-critique-pattern.md

Chapter Overview

One of the most important discoveries in Agentic AI is that a model can often improve its own results if given an opportunity to review, critique, and revise its work.

This concept forms the foundation of the **Reflection Pattern**.

Instead of:

1	Generate
2	↓
3	Return Answer

a reflection-based system follows:

1	Generate
2	↓
3	Critique
4	↓
5	Revise
6	↓
7	Validate
8	↓
9	Return Answer

The Reflection Pattern is one of the highest-impact architectural improvements available for:

- Code generation
- SQL generation
- Compliance reviews
- Research assistants

- Report generation
- Planning systems
- Multi-agent workflows
- RAG applications

Modern enterprise systems frequently combine:

1	ReAct
2	+
3	Reflection
4	+
5	RAG
6	+
7	Governance

to achieve substantially higher accuracy.

Learning Objectives

After completing this chapter you should be able to:

- Understand the Reflection Pattern
 - Design self-critiquing agents
 - Build iterative improvement loops
 - Implement reflection in LangGraph
 - Measure reflection effectiveness
 - Avoid reflection anti-patterns
 - Control reflection costs
 - Apply reflection safely in production
 - Explain reflection architectures during architect interviews
-

Prerequisites

Required:

- Chapter 1: Generative AI Foundations
- Chapter 2: Agentic AI Fundamentals
- Chapter 3: Agent Design Principles
- Chapter 4: ReAct Pattern
- Chapter 5: Router Pattern

1. Introduction

Most AI systems operate like this:

1	Question
2	↓
3	Answer

Unfortunately:

1	First Answer
2	≠
3	Best Answer

Example:

Request:

1	Write a SQL query that identifies
2	duplicate customer accounts.

Generated SQL:

1	SELECT *
2	FROM customer;

Clearly incorrect.

Traditional systems return the response.

Reflection systems ask:

1	Is this answer correct?
---	-------------------------

before responding.

This extra step significantly improves quality.

2. Core Concepts

What Is Reflection?

Definition

Reflection is an architectural pattern where an agent evaluates its own output before finalizing a response.

The system effectively performs:

- 1 Self Review
- 2 Self Critique
- 3 Self Improvement

Core Principle

- 1 Generate
- 2 ↓
- 3 Review
- 4 ↓
- 5 Improve

instead of:

- 1 Generate
- 2 ↓
- 3 Ship

3. Why Reflection Exists

Large Language Models commonly fail due to:

- Missing information
- Poor reasoning
- Hallucinations
- Incomplete answers
- Logical mistakes

Reflection adds a second reasoning cycle.

Question



Generation



Critique



Revision



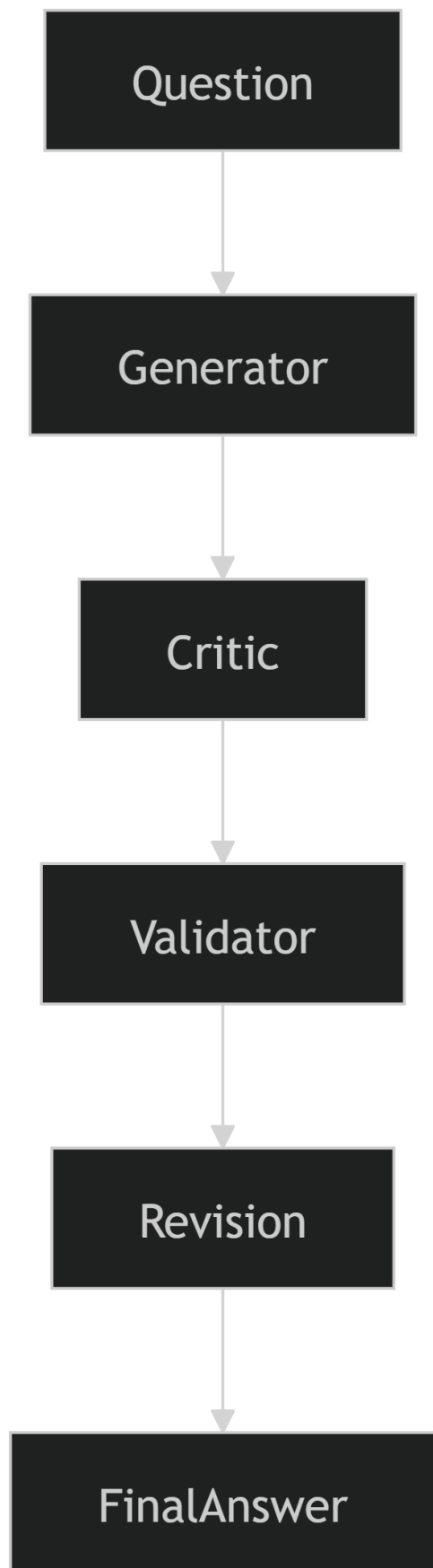
Answer

This frequently improves quality dramatically.

4. Reflection Architecture

Basic Reflection Pattern

Enterprise Reflection Pattern



The separation of responsibilities improves reliability.

5. Types of Reflection

Type 1: Self Reflection

Same model performs:

- Generation
- Critique
- Revision

Architecture:

```
1  Model
2  ↓
3  Generate
4
5  Model
6  ↓
7  Review
8
9  Model
10 ↓
11 Improve
```

Simplest implementation.

Type 2: Dedicated Critic Agent

Separate reviewer.

```
1  Generator
2    ↓
3  Critic
4    ↓
5  Generator
```

Common in multi-agent systems.

Type 3: Governance Reflection

Evaluate output against policies.

Example:

1 Compliance Review

before release.

Type 4: Evidence Reflection

Evaluate grounding.

Question:

1 Do retrieved documents support this answer?

Very useful for RAG systems.

6. Reflection Lifecycle

Stage 1

Generate Initial Result

1 Draft

Stage 2

Identify Weaknesses

Examples:

1 Missing citations
2
3 Incorrect assumptions
4
5 Incomplete reasoning

Stage 3

Improve Output

1 Fix issues

Stage 4

Validate

1 Ready?

Stage 5

Return Final Response

7. Reflection Evaluation Dimensions

Accuracy

1 Is it correct?

Completeness

1 Is anything missing?

Consistency

1 Does logic align?

Evidence

1 Can claims be supported?

Compliance

1 Does output satisfy policy?

8. Reflection in Code Generation

Initial Output

1 users = db.query()

```
2 print(users)
```

Reflection

Critique:

```
1 No exception handling.  
2  
3 No pagination.  
4  
5 No logging.
```

Revised Output

```
1 try:  
2     users = db.query(limit=100)  
3  
4     logger.info(  
5         "Retrieved users"  
6     )  
7  
8 except Exception as e:  
9     logger.error(str(e))
```

Reflection improved quality substantially.

9. Reflection in SQL Generation

Initial query:

```
1 SELECT *  
2 FROM customer
```

Critique:

```
1 No filters.  
2  
3 Returns excessive data.  
4  
5 No indexing considerations.
```

Improved query:

```
1  SELECT customer_id,  
2         customer_name  
3  FROM customer  
4  WHERE is_active = TRUE
```

10. Reflection in RAG

Reflection is especially valuable in Retrieval-Augmented Generation.

Process:

Question



Retrieval



Answer



Reflection



Validate



Final

Reflection questions:

- 1 Did retrieval answer question?
- 2
- 3 Was evidence sufficient?
- 4
- 5 Are citations valid?

11. Reflection and Hallucination Reduction

One major purpose of reflection:

- 1 Detect Unsupported Claims

Example

Generated:

- 1 Policy updated in 2025.

Reflection:

- 1 Evidence not found.

Revision:

- 1 Unable to verify policy update date.

Much safer response.

12. Reflection and Planning

Reflection can improve planning systems.

Initial Plan:

- 1 Task A
- 2 Task B

Critique:

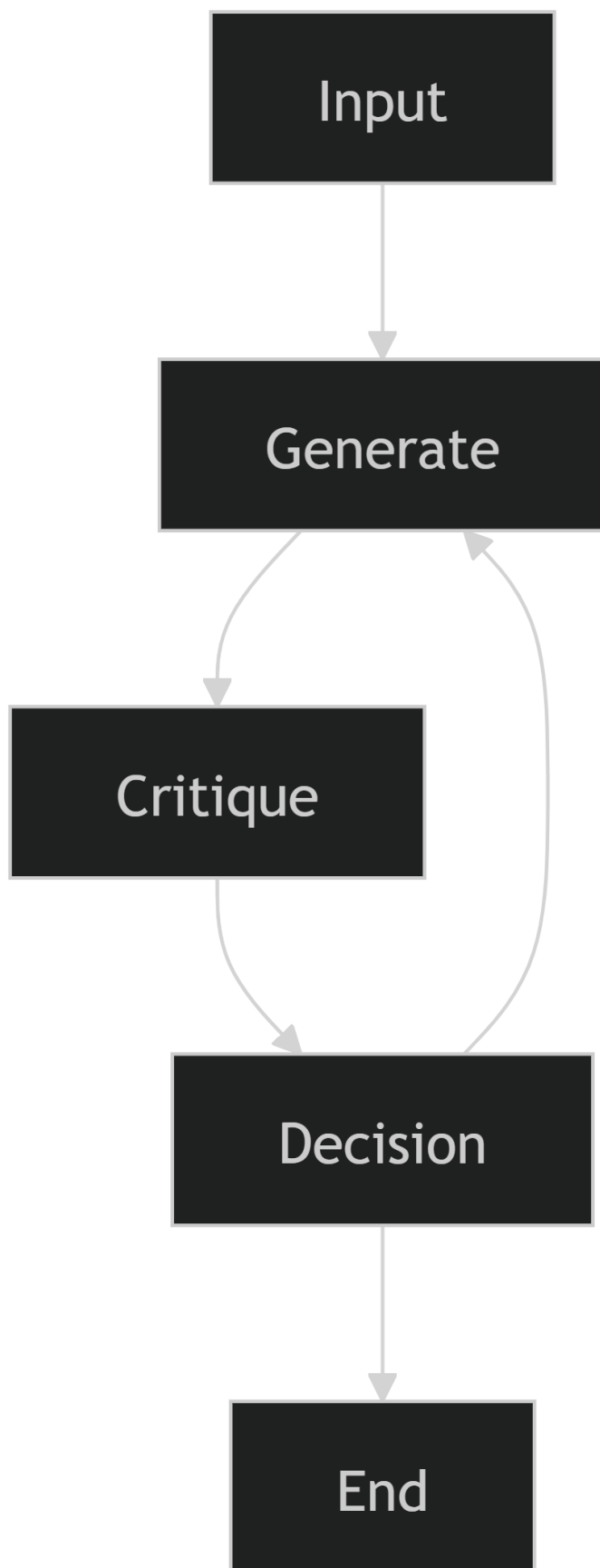
- 1 Missing dependency analysis.

Updated Plan:

- 1 Dependency Review
- 2 Task A
- 3 Task B

13. LangGraph Reflection Architecture

Graph Design



State Model

```
1 from typing import TypedDict
2
3 class ReflectionState(TypedDict):
4     question: str
5     draft: str
6     critique: str
7     final_answer: str
```

Generate Node

```
1 def generate(state):
2     return {
3         "draft": llm.invoke(
4             state["question"]
5         )
6     }
```

Critique Node

```
1 def critique(state):
2     return {
3         "critique": critique_llm(
4             state["draft"]
5         )
6     }
```

Decision Node

```
1 def should_revise(state):
2
3     if "major issue" in state["critique"]:
4         return "generate"
5
6     return "end"
```

14. Multi-Agent Reflection

Architecture:

Specialized Reviewers

Examples:

1	Security Reviewer
2	
3	Compliance Reviewer
4	
5	Financial Reviewer
6	
7	Technical Reviewer

Useful for high-risk outputs.

15. CrewAI Reflection Pattern

1	Worker Agent
2	↓
3	Reviewer Agent
4	↓
5	Improved Result

Frequently implemented through separate specialized agents.

16. AutoGen Reflection Pattern

1	Writer Agent
2	↓
3	Critic Agent
4	↓
5	Writer Agent

One of AutoGen's strongest workflows.

17. Semantic Kernel Reflection

Common flow:

1	Generate
2	↓
3	Evaluate
4	↓
5	Improve

Often integrated into planner-based architectures.

18. Enterprise Production Example

Compliance Assistant

Question:

1	Does this process satisfy PCI controls?
---	---

Flow:

1	Answer
2	↓
3	Compliance Critique
4	↓
5	Evidence Verification
6	↓
7	Revised Answer

This dramatically reduces risk.

19. Enterprise Research Example

Question:

1	Create executive market analysis.
---	-----------------------------------

Reflection evaluates:

- Data gaps
- Unsupported conclusions
- Missing evidence
- Contradictions

20. Cost Considerations

Reflection increases:

- Tokens
 - Latency
 - Complexity
-

Single Pass

1	1x Cost
---	---------

Reflection

1	2-4x Cost
---	-----------

Therefore:

Use selectively.

21. Scalability

Reflection should not be applied universally.

Recommended:

Output Type	Reflection
FAQ	No
Search Results	Limited
Compliance	Yes
Legal	Yes
Financial	Yes
Code Generation	Yes

22. Reliability

Reflection improves:

- Accuracy

- Consistency
- Explainability

But creates:

- Cost
- Latency
- Additional failure points

Must be monitored.

23. Security

Reflection can also evaluate:

Prompt Injection

1 Potential Prompt Injection Detected

Tool Abuse

1 Unsafe Tool Request

Data Leakage

1 Sensitive Information Exposed

Reflection becomes a security layer.

24. Observability

Track:

- Reflection frequency
- Revision count
- Critique quality
- Hallucination reduction
- Cost increase
- Latency increase

Example metrics:

1	avg_reflections
2	
3	revisions_per_task
4	
5	reflection_success_rate

25. Common Mistakes

Mistake 1

Reflecting every answer.

Mistake 2

Unlimited reflection cycles.

Mistake 3

Weak critique prompts.

Mistake 4

No objective evaluation criteria.

Mistake 5

Ignoring reflection cost.

26. Anti-Patterns

Infinite Reflection

1	Generate
2	↓
3	Reflect
4	↓
5	Generate

6	↓
7	Reflect

Never ends.

Self-Congratulation Pattern

Model always validates itself.

No value.

Reflection Without Evidence

Critique becomes subjective.

Reflection Instead Of Testing

Reflection supplements quality assurance.

It does not replace testing.

27. Trade-Offs

Benefit	Cost
Better quality	More tokens
Better reasoning	Higher latency
Fewer hallucinations	More complexity
Better governance	More infrastructure

28. Architecture Decision Matrix

Scenario	Reflection Fit
SQL Generation	Excellent
Code Generation	Excellent
Compliance Review	Excellent
RAG Systems	Excellent
Legal Analysis	Excellent
FAQ Bot	Poor
Deterministic Workflow	Poor

29. Real-World Use Cases

Code Assistants

Review generated code before delivery.

Compliance Platforms

Review control assessments.

Enterprise Search

Validate retrieved evidence.

Legal Review Systems

Evaluate document interpretations.

Agentic Operations Platforms

Improve remediation recommendations.

30. Interview Questions

Q1

What is the Reflection Pattern?

Q2

How does reflection improve accuracy?

Q3

Why is reflection useful in RAG?

Q4

What are the biggest risks?

Q5

How is reflection implemented in LangGraph?

Q6

How does reflection compare to ReAct?

Q7

When should reflection be avoided?

Q8

What enterprise systems benefit most?

31. Interview Answers

Q1

A pattern where agents review and improve their own outputs before finalizing responses.

Q2

By identifying errors, omissions, unsupported claims, and logical weaknesses.

Q3

It validates retrieval quality and evidence grounding.

Q4

Infinite loops, excessive cost, weak critiques, and latency.

Q5

Through generate → critique → revise graph structures.

Q6

ReAct gathers information. Reflection improves outputs.

Q7

For simple, deterministic, low-risk tasks.

Q8

Compliance, legal, code generation, RAG, and research systems.

32. Architecture Review Checklist

Reflection Readiness

- Reflection objective defined
- Critique dimensions defined
- Revision criteria documented
- Loop limits configured

- Cost budget defined
 - Observability implemented
 - Security review completed
 - Governance integrated
-

33. Practical Exercises

1. Design a reflection workflow for code review.
 2. Build a LangGraph reflection loop.
 3. Compare ReAct and Reflection architectures.
 4. Design a compliance reviewer agent.
 5. Measure quality improvements from reflection.
-

34. Key Takeaways

- Reflection allows agents to improve their own outputs.
 - Reflection significantly reduces hallucinations.
 - Reflection works especially well for high-risk outputs.
 - ReAct gathers evidence; Reflection improves responses.
 - Multi-agent review systems often outperform self-reflection.
 - LangGraph supports reflection naturally.
 - Reflection is expensive and should be used selectively.
 - Infinite reflection is a major anti-pattern.
 - Reflection can enhance governance and security.
 - High-quality enterprise agents frequently combine ReAct, RAG, Reflection, and Governance.
-

35. References

- Reflexion: Language Agents with Verbal Reinforcement Learning
- ReAct Paper
- LangGraph Documentation
- CrewAI Documentation
- AutoGen Documentation
- Semantic Kernel Documentation

- Enterprise Agent Evaluation Patterns
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 8 – Corrective RAG Pattern (Self-Correcting Retrieval and Adaptive Knowledge Access)